SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Week 7: Task C.4

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY – FEB-2026

Contents

1. Overview of what I implemented 3
2. Dynamic DL Model Builder Function (Requirement 1) 3
2.1 Purpose 3
2.2 Function inputs and outputs (what it takes and returns) 3
2.3 Less straightforward lines (key code reasoning) 5
2.4 References used 5
3. Experiment Setup 6
3.1 Controlled settings (kept constant) 6
3.2 Variables tested 6
3.3 Evaluation metrics 6
4. Results Summary (6 runs) 7
4.1 Config A — epochs=25, batch_size=64 7
4.2 Config B — epochs=32, batch_size=50 7
5. Discussion 7
5.1 Accuracy comparison 7
5.2 Training speed comparison 7
5.3 Effect of changing batch size and epochs 8
6. Conclusion 8

1. Overview of what I implemented

After completing Task C.2 and Task C.3, the code base was upgraded to version v0.3 to improve how Deep Learning (DL) networks are constructed and tested. In v0.1/v0.2, the model architecture was manually written by explicitly stacking layers. For Task C.4, I implemented a dynamic model-building function that can generate different recurrent neural network architectures based on parameters such as:

- layer type (LSTM / GRU / RNN)
- number of layers
- units per layer
- dropout rate

Using this function, I conducted experiments across different DL network types and training hyperparameters (epochs and batch size), then evaluated performance using MAE/MSE/RMSE on the test set.2. Candlestick chart function (with n-day aggregation option)

2. Dynamic DL Model Builder Function (Requirement 1)

2.1 Purpose

The goal of the model builder is to avoid hard-coding a single architecture and enable controlled experimentation. Instead of writing separate model definitions for LSTM, GRU, and RNN, I implemented one function that creates a compiled Keras model from input parameters.

2.2 Function inputs and outputs (what it takes and returns)

Inputs:

- layer_name: string indicating the recurrent layer type ("LSTM", "GRU", or "RNN")
- input_shape: (lookback_days, n_features) (in my project: (60, 5) using OHLCV)
- num_layers: number of stacked recurrent layers (e.g., 3)
- units: either a single integer (same units for every layer) or a list like [50, 50, 50]
- dropout: dropout rate applied after recurrent layers
- compile settings: optimizer (adam) and loss (mean_squared_error)

```python
# Build the Model (Task C.4 v0.3)
# -------------------------------------------------------

n_features = x_train.shape[2]
input_shape = (PREDICTION_DAYS, n_features)

MODEL_TYPE = "GRU"          # try: "LSTM", "GRU", "RNN"
NUM_LAYERS = 3
UNITS = [50, 50, 50]         # or just 50
DROPOUT = 0.2

n_features = x_train.shape[2]
input_shape = (PREDICTION_DAYS, n_features)



# n_features = x_train.shape[2]
# input_shape = (PREDICTION_DAYS, n_features)



model = build_recurrent_model(
    layer_name=MODEL_TYPE,
    input_shape=input_shape,
    num_layers=NUM_LAYERS,
    units=UNITS,
    dropout=DROPOUT,
    dense_units=1,
    optimizer="adam",
    loss="mean_squared_error"
)
```

Output:

- A compiled Keras Sequential model that can be trained immediately.

```
Model type: GRU
Model: "GRU_3layers"
```

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 60, 50)	8,550
dropout (Dropout)	(None, 60, 50)	0
gru_1 (GRU)	(None, 60, 50)	15,300
dropout_1 (Dropout)	(None, 60, 50)	0
gru_2 (GRU)	(None, 50)	15,300
dropout_2 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

```
Total params: 39,201 (153.13 KB)
Trainable params: 39,201 (153.13 KB)
Non-trainable params: 0 (0.00 B)
```

```
Train time (sec): 16.739345999987563
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step
Test MAE: 2.891448446166464
Test MSE: 12.157420018005983
Test RMSE: 3.486749204919388
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
Next-day prediction for CBA.AX: [[113.804306]]
Test MAE: 2.891448446166464
Test MSE: 12.157420018005983
Test RMSE: 3.486749204919388
```

2.3 Less straightforward lines (key code reasoning)

(a) Mapping layer_name to the correct Keras class

To support multiple DL networks, I used a mapping that converts a user-friendly string (e.g., "GRU") into the corresponding Keras layer class (GRU). This allows the same loop to build different model families without duplicating code.

```
MODEL_TYPE = "GRU"          # try: "LSTM", "GRU", "RNN"
```

(b) Handling "units" as either int or list

The function supports:

- units = 50 → becomes [50, 50, 50] for 3 layers
- units = [50, 50, 50] → uses each value per layer

```
NUM_LAYERS = 3
UNITS = [50, 50, 50]         # or just 50
```

This makes it easy to test unequal layer sizes in future experiments.

(c) return_sequences logic for stacked recurrent layers

The most important stacked-RNN rule is:

- Intermediate recurrent layers must use return_sequences=True
- The last recurrent layer uses return_sequences=False (default)

```
    for i in range(num_layers):
        return_seq = (i < num_layers - 1)  # True except last recurrent layer

        if i == 0:
            model.add(RNNLayer(units=units_list[i], return_sequences=return_seq, input_shape=input_shape))
        else:
            model.add(RNNLayer(units=units_list[i], return_sequences=return_seq))

        if dropout and dropout > 0:
            model.add(Dropout(dropout))
```

Reason: When stacking recurrent layers, the next layer expects a sequence input (timesteps × features). If return_sequences=False is used too early, the sequence collapses to a single vector and cannot be fed into another recurrent layer.

(d) Dense(1) output for regression

Stock prediction here is regression (predicting a continuous price), so the output layer is Dense(1).

2.4 References used

- Keras documentation / tutorials on LSTM/GRU/SimpleRNN and return_sequences behavior:
    - Official Keras RNN API Reference: keras.io/api/layers/recurrent_layers/
    - Official Keras RNN API Reference: keras.io/api/layers/recurrent_layers/

3. Experiment Setup

3.1 Controlled settings (kept constant)

To make model comparisons fair, I kept the following fixed:

- Stock: CBA.AX
- Features: Open, High, Low, Close, Volume (5 features)
- Lookback window: 60 days
- Same data preprocessing/scaling and same train/test split
- Same architecture depth and units:
  - num_layers = 3
  - units = [50, 50, 50]
  - dropout = 0.2
- Same optimizer and loss: Adam + MSE.

```
n_features = x_train.shape[2]
input_shape = (PREDICTION_DAYS, n_features)

MODEL_TYPE = "GRU"          # try: "LSTM", "GRU", "RNN"
NUM_LAYERS = 3
UNITS = [50, 50, 50]         # or just 50
DROPOUT = 0.2

n_features = x_train.shape[2]
input_shape = (PREDICTION_DAYS, n_features)
```

3.2 Variables tested

I tested:

- Model types: LSTM vs GRU vs RNN (SimpleRNN)
- Training hyperparameters: two configurations
  - Config A: epochs = 25, batch_size = 64
  - Config B: epochs = 32, batch_size = 50

```
# ----------------------------
# Training (timed) - Task C.4 experiments
# ----------------------------
EPOCHS = 25
BATCH_SIZE = 64
```

3.3 Evaluation metrics

I evaluated performance using:

- MAE (Mean Absolute Error)
- MSE
- RMSE

Lower values indicate better predictive accuracy. I also recorded training time (seconds) per run.

```
Train time (sec): 16.739345999987563
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step
Test MAE: 2.891448446166464
Test MSE: 12.157420018005983
Test RMSE: 3.486749204919388
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
Next-day prediction for CBA.AX: [[113.804306]]
Test MAE: 2.891448446166464
Test MSE: 12.157420018005983
Test RMSE: 3.486749204919388
```

4. Results Summary (6 runs)

4.1 Config A — epochs=25, batch_size=64

Model	Params	Train time (s)	Test MAE	Test MSE	Test RMSE
LSTM (3×50)	51,651	14.24	5.770	47.206	6.871
GRU (3×50)	39,201	16.74	**2.891**	**12.157**	**3.487**
RNN (3×50)	12,951	**6.95**	4.280	29.543	5.435

4.2 Config B — epochs=32, batch_size=50

Model	Params	Train time (s)	Test MAE	Test MSE	Test RMSE
LSTM (3×50)	51,651	19.41	3.826	22.658	4.760
GRU (3×50)	39,201	21.59	**2.782**	**11.210**	**3.348**
RNN (3×50)	12,951	**9.39**	4.622	29.172	5.401

5. Discussion

5.1 Accuracy comparison

Across both configurations, GRU achieved the best accuracy (lowest MAE/MSE/RMSE).

- Best overall run: GRU (epochs=32, batch=50) with MAE 2.782 and RMSE 3.348.

5.2 Training speed comparison

The RNN (SimpleRNN) model trained the fastest in both settings due to the lowest parameter count, but it consistently had worse accuracy than GRU.

5.3 Effect of changing batch size and epochs

Changing training hyperparameters affected models differently:

- LSTM improved significantly when moving from Config A to Config B (MAE 5.770 → 3.826).
- GRU improved slightly (MAE 2.891 → 2.782).
- RNN did not improve and slightly worsened in MAE.

This suggests that hyperparameter sensitivity exists and should be tuned per architecture.

6. Conclusion

Task C.4 successfully upgraded the project to v0.3 by introducing a dynamic model creation function that supports multiple recurrent architectures. Using this function, I performed controlled experiments across LSTM, GRU, and RNN with two training configurations. Based on MAE/MSE/RMSE results, GRU (3 layers × 50 units) produced the most accurate predictions for CBA.AX, while SimpleRNN trained the fastest but with lower accuracy.